# Stat405 Bayesian Coding Part

Aiwei Yang42274209, Chang Sun 19062439

2026-04-17

1. Read and prepare data

```r
# Read dataset
df <- read_csv("day2018.csv", show_col_types = FALSE)

# Convert selected variables to factors
bike_clean <- df %>%
  mutate(
    season = factor(season),
    yr = factor(yr),
    workingday = factor(workingday),
    holiday = factor(holiday)
  )
# Standardize continuous predictors
bike_clean <- bike_clean %>%
  mutate(
    temp_std = as.numeric(scale(temp)),
    hum_std = as.numeric(scale(hum)),
    windspeed_std = as.numeric(scale(windspeed))
  )
# Quick check
glimpse(bike_clean)
```

```
## Rows: 730
## Columns: 19
## $ instant       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
## $ dteday        <chr> "01-01-2018", "02-01-2018", "03-01-2018", "04-01-2018", ~
## $ season        <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ yr            <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ mnth          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ holiday       <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,~
## $ weekday       <dbl> 6, 0, 1, 2, 3, 4, 5, 6, 0, 1, 2, 3, 4, 5, 6, 0, 1, 2, 3,~
## $ workingday    <fct> 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1,~
## $ weathersit    <dbl> 2, 2, 1, 1, 1, 1, 2, 2, 1, 1, 2, 1, 1, 1, 2, 1, 2, 2, 2,~
## $ temp          <dbl> 14.110847, 14.902598, 8.050924, 8.200000, 9.305237, 8.37~
## $ atemp         <dbl> 18.181250, 17.686950, 9.470250, 10.606100, 11.463500, 11~
## $ hum           <dbl> 80.5833, 69.6087, 43.7273, 59.0435, 43.6957, 51.8261, 49~
## $ windspeed     <dbl> 10.749882, 16.652113, 16.636703, 10.739832, 12.522300, 6~
## $ casual        <dbl> 331, 131, 120, 108, 82, 88, 148, 68, 54, 41, 43, 25, 38,~
## $ registered    <dbl> 654, 670, 1229, 1454, 1518, 1518, 1362, 891, 768, 1280, ~
## $ cnt           <dbl> 985, 801, 1349, 1562, 1600, 1606, 1510, 959, 822, 1321, ~
## $ temp_std      <dbl> -0.8270463, -0.7215741, -1.6343117, -1.6144528, -1.46721~
```

```
## $ hum_std       <dbl> 1.25148472, 0.48066601, -1.33715581, -0.26139783, -1.339~
## $ windspeed_std <dbl> -0.38756724, 0.74838579, 0.74541996, -0.38950148, -0.046~
```

```r
summary(bike_clean)
```

```
##     instant         dteday          season  yr           mnth        holiday
##  Min.   :  1.0   Length:730         1:180   0:365   Min.   : 1.000   0:709
##  1st Qu.:183.2   Class :character   2:184   1:365   1st Qu.: 4.000   1: 21
##  Median :365.5   Mode  :character   3:188           Median : 7.000
##  Mean   :365.5                      4:178           Mean   : 6.526
##  3rd Qu.:547.8                                      3rd Qu.:10.000
##  Max.   :730.0                                      Max.   :12.000
##     weekday      workingday   weathersit         temp            atemp
##  Min.   :0.000   0:231      Min.   :1.000   Min.   : 2.424   Min.   : 3.953
##  1st Qu.:1.000   1:499      1st Qu.:1.000   1st Qu.:13.812   1st Qu.:16.890
##  Median :3.000              Median :1.000   Median :20.466   Median :24.368
##  Mean   :2.997              Mean   :1.395   Mean   :20.319   Mean   :23.726
##  3rd Qu.:5.000              3rd Qu.:2.000   3rd Qu.:26.881   3rd Qu.:30.446
##  Max.   :6.000              Max.   :3.000   Max.   :35.328   Max.   :42.045
##       hum          windspeed          casual         registered
##  Min.   : 0.00   Min.   : 1.500   Min.   :   2.0   Min.   :  20
##  1st Qu.:52.00   1st Qu.: 9.042   1st Qu.: 316.2   1st Qu.:2502
##  Median :62.62   Median :12.125   Median : 717.0   Median :3664
##  Mean   :62.77   Mean   :12.764   Mean   : 849.2   Mean   :3659
##  3rd Qu.:72.99   3rd Qu.:15.626   3rd Qu.:1096.5   3rd Qu.:4783
##  Max.   :97.25   Max.   :34.000   Max.   :3410.0   Max.   :6946
##       cnt          temp_std           hum_std          windspeed_std
##  Min.   :  22   Min.   :-2.38385   Min.   :-4.408413   Min.   :-2.1678
##  1st Qu.:3170   1st Qu.:-0.86687   1st Qu.:-0.756109   1st Qu.:-0.7163
##  Median :4548   Median : 0.01952   Median :-0.009845   Median :-0.1228
##  Mean   :4508   Mean   : 0.00000   Mean   : 0.000000   Mean   : 0.0000
##  3rd Qu.:5966   3rd Qu.: 0.87406   3rd Qu.: 0.718127   3rd Qu.: 0.5508
##  Max.   :8714   Max.   : 1.99942   Max.   : 2.422097   Max.   : 4.0872
```

2. Build design matrix

```r
X <- model.matrix(
  cnt ~ season + yr + temp_std + hum_std + windspeed_std + workingday + holiday,
  data = bike_clean
)
dim(X)
```

```
## [1] 730  10
```

```r
head(X)
```

```
##   (Intercept) season2 season3 season4 yr1   temp_std    hum_std windspeed_std
## 1           1       0       0       0   0 -0.8270463  1.2514847   -0.38756724
## 2           1       0       0       0   0 -0.7215741  0.4806660    0.74838579
## 3           1       0       0       0   0 -1.6343117 -1.3371558    0.74541996
## 4           1       0       0       0   0 -1.6144528 -0.2613978   -0.38950148
## 5           1       0       0       0   0 -1.4672199 -1.3393753   -0.04644478
```

```
## 6           1       0       0       0   0 -1.5907050 -0.7683235   -1.30157017
##   workingday1 holiday1
## 1           0        0
## 2           0        0
## 3           1        0
## 4           1        0
## 5           1        0
## 6           1        0
```

3. Prepare data for Stan

```r
stan_data <- list(
  N = nrow(X),
  K = ncol(X),
  y = bike_clean$cnt,
  X = X
)

stan_data$N
```

```
## [1] 730
```

```r
stan_data$K
```

```
## [1] 10
```

```r
head(stan_data$y)
```

```
## [1]  985  801 1349 1562 1600 1606
```

4. Remove old stan file if they exist

```r
if (file.exists("poisson_model.stan")) file.remove("poisson_model.stan")
if (file.exists("negbin_model.stan")) file.remove("negbin_model.stan")
```

5. Write Poisson Stan model

```r
poisson_code <- "
data {
  int<lower=1> N;
  array[N] int<lower=0> y;
  int<lower=1> K;
  matrix[N, K] X;
}
parameters {
  vector[K] beta;
}
model {
  beta ~ normal(0, 2.5);
  y ~ poisson_log(X * beta);
```

```
}
generated quantities {
  vector[N] log_lik;
  array[N] int y_rep;

  for (n in 1:N) {
    log_lik[n] = poisson_log_lpmf(y[n] | X[n] * beta);
    y_rep[n] = poisson_log_rng(X[n] * beta);
  }
}
"

writeLines(poisson_code, "poisson_model.stan")

# Verify the file contents
readLines("poisson_model.stan")
```

```
##  [1] ""
##  [2] "data {"
##  [3] "  int<lower=1> N;"
##  [4] "  array[N] int<lower=0> y;"
##  [5] "  int<lower=1> K;"
##  [6] "  matrix[N, K] X;"
##  [7] "}"
##  [8] "parameters {"
##  [9] "  vector[K] beta;"
## [10] "}"
## [11] "model {"
## [12] "  beta ~ normal(0, 2.5);"
## [13] "  y ~ poisson_log(X * beta);"
## [14] "}"
## [15] "generated quantities {"
## [16] "  vector[N] log_lik;"
## [17] "  array[N] int y_rep;"
## [18] ""
## [19] "  for (n in 1:N) {"
## [20] "    log_lik[n] = poisson_log_lpmf(y[n] | X[n] * beta);"
## [21] "    y_rep[n] = poisson_log_rng(X[n] * beta);"
## [22] "  }"
## [23] "}"
## [24] ""
```

6. Compile Poisson model

```
poisson_mod <- cmdstan_model("poisson_model.stan")
```

7. Fit Poisson model using HMC

```
fit_poisson_hmc <- poisson_mod$sample(
  data = stan_data,
  chains = 4,
  parallel_chains = 4,
```

```r
  iter_warmup = 1500,
  iter_sampling = 1500,
  adapt_delta = 0.99,
  max_treedepth = 15,
  seed = 123,
  refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 1.5 seconds.
## Chain 3 finished in 1.5 seconds.
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 1.6 seconds.
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 1.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 1.6 seconds.
## Total execution time: 1.8 seconds.
```

```r
poisson_hmc_summary <- fit_poisson_hmc$summary()
poisson_hmc_summary
```

```
## # A tibble: 1,471 x 10
##    variable       mean   median      sd     mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__        2.47e+7  2.47e+7 2.25e+0 1.48e+0  2.47e+7  2.47e+7 1.00     2421.
##  2 beta[1]     7.84e+0  7.84e+0 2.19e-3 2.16e-3  7.84e+0  7.84e+0 1.00     1716.
##  3 beta[2]     3.32e-1  3.32e-1 2.33e-3 2.30e-3  3.28e-1  3.35e-1 1.00     1561.
##  4 beta[3]     2.28e-1  2.28e-1 2.91e-3 2.86e-3  2.23e-1  2.33e-1 1.00     1529.
##  5 beta[4]     4.52e-1  4.52e-1 2.02e-3 2.01e-3  4.49e-1  4.55e-1 1.00     1904.
##  6 beta[5]     4.60e-1  4.60e-1 1.14e-3 1.16e-3  4.58e-1  4.61e-1 1.00     5116.
##  7 beta[6]     2.43e-1  2.43e-1 1.04e-3 1.00e-3  2.41e-1  2.45e-1 1.00     2472.
##  8 beta[7]    -8.70e-2 -8.69e-2 6.19e-4 6.18e-4 -8.80e-2 -8.59e-2 1.00     8224.
##  9 beta[8]    -6.24e-2 -6.24e-2 6.12e-4 6.17e-4 -6.34e-2 -6.14e-2 1.000    8887.
## 10 beta[9]     1.87e-2  1.87e-2 1.24e-3 1.25e-3  1.67e-2  2.08e-2 1.00     4467.
## # i 1,461 more rows
## # i 1 more variable: ess_tail <dbl>
```

8. Inspect Poisson HMC diagnostics

```r
# Print summary table
poisson_hmc_summary
```

```
## # A tibble: 1,471 x 10
##    variable       mean   median      sd     mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__        2.47e+7  2.47e+7 2.25e+0 1.48e+0  2.47e+7  2.47e+7 1.00     2421.
##  2 beta[1]     7.84e+0  7.84e+0 2.19e-3 2.16e-3  7.84e+0  7.84e+0 1.00     1716.
##  3 beta[2]     3.32e-1  3.32e-1 2.33e-3 2.30e-3  3.28e-1  3.35e-1 1.00     1561.
##  4 beta[3]     2.28e-1  2.28e-1 2.91e-3 2.86e-3  2.23e-1  2.33e-1 1.00     1529.
##  5 beta[4]     4.52e-1  4.52e-1 2.02e-3 2.01e-3  4.49e-1  4.55e-1 1.00     1904.
##  6 beta[5]     4.60e-1  4.60e-1 1.14e-3 1.16e-3  4.58e-1  4.61e-1 1.00     5116.
##  7 beta[6]     2.43e-1  2.43e-1 1.04e-3 1.00e-3  2.41e-1  2.45e-1 1.00     2472.
##  8 beta[7]    -8.70e-2 -8.69e-2 6.19e-4 6.18e-4 -8.80e-2 -8.59e-2 1.00     8224.
##  9 beta[8]    -6.24e-2 -6.24e-2 6.12e-4 6.17e-4 -6.34e-2 -6.14e-2 1.000    8887.
## 10 beta[9]     1.87e-2  1.87e-2 1.24e-3 1.25e-3  1.67e-2  2.08e-2 1.00     4467.
## # i 1,461 more rows
## # i 1 more variable: ess_tail <dbl>
```

```r
# Check the largest R-hat value
max(poisson_hmc_summary$rhat, na.rm = TRUE)
```

```
## [1] 1.003341
```

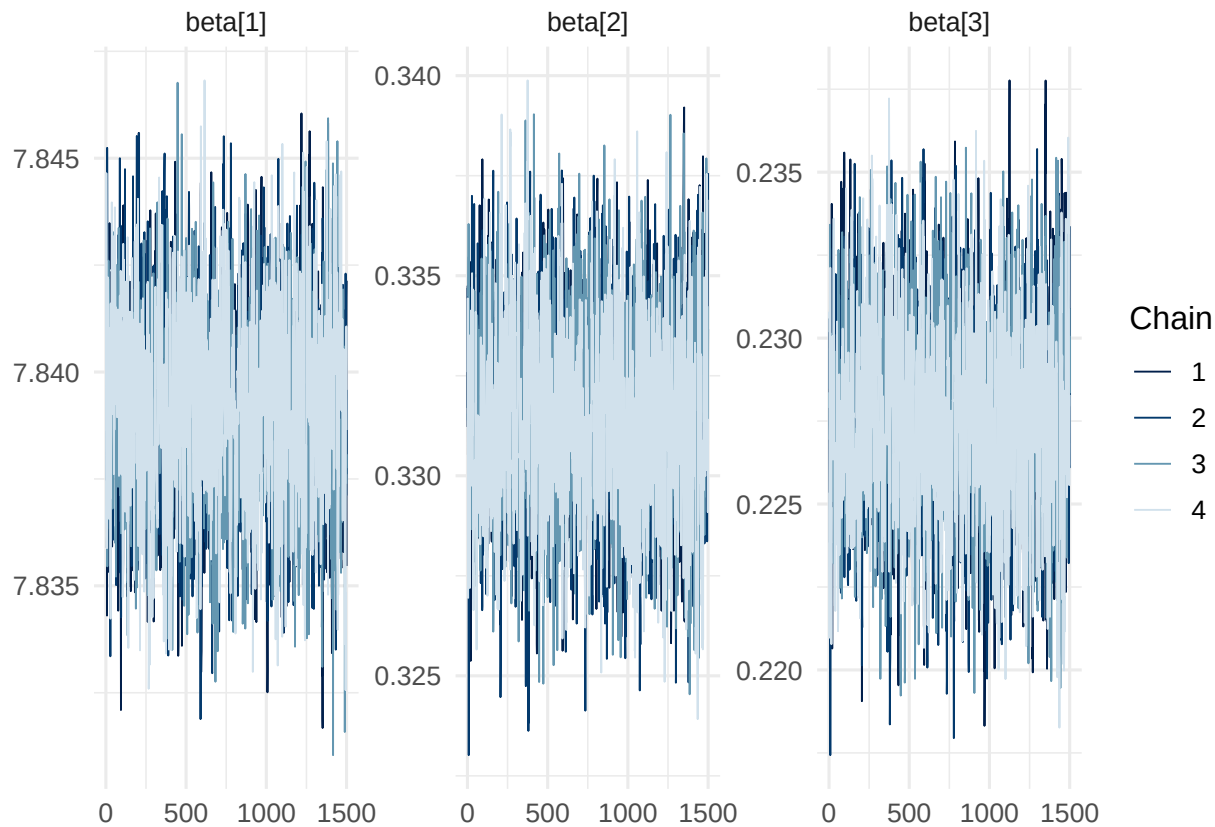
```r
# Check the smallest bulk ESS
min(poisson_hmc_summary$ess_bulk, na.rm = TRUE)
```

```
## [1] 1529.346
```

```r
# Check the smallest tail ESS
min(poisson_hmc_summary$ess_tail, na.rm = TRUE)
```

```
## [1] 1831.3
```

```r
# Traceplots for selected Poisson coefficients
mcmc_trace(
  fit_poisson_hmc$draws(variables = c("beta[1]", "beta[2]", "beta[3]"))
)
```




9. Fit Poisson model using Variational Inference

```r
fit_poisson_vi <- poisson_mod$variational(
  data = stan_data,
  algorithm = "meanfield",
  iter = 20000,
  output_samples = 2000,
  seed = 123
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 1.8e-05 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 0.18 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
```

```
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100     -2687516.618             1.000            1.000
##    200      -446077.710             3.012            5.025
##    300      -101285.998             3.143            3.404
##    400      -112202.212             2.382            3.404
##    500       -87860.669             1.961            1.000
##    600       -82139.872             1.645            1.000
##    700       -82847.598             1.412            0.277
##    800       -94800.546             1.251            0.277
##    900       -89769.916             1.118            0.126
##   1000       -86413.230             1.010            0.126
##   1100       -89462.231             0.921            0.097   MAY BE DIVERGING... INSPECT ELBO
##   1200       -86232.827             0.848            0.097   MAY BE DIVERGING... INSPECT ELBO
##   1300       -87128.074             0.783            0.070   MAY BE DIVERGING... INSPECT ELBO
##   1400       -87795.030             0.728            0.070   MAY BE DIVERGING... INSPECT ELBO
##   1500       -86619.860             0.680            0.056   MAY BE DIVERGING... INSPECT ELBO
##   1600       -84381.965             0.639            0.056   MAY BE DIVERGING... INSPECT ELBO
##   1700       -88291.397             0.604            0.044   MAY BE DIVERGING... INSPECT ELBO
##   1800       -85620.870             0.573            0.044   MAY BE DIVERGING... INSPECT ELBO
##   1900       -82841.597             0.544            0.039   MAY BE DIVERGING... INSPECT ELBO
##   2000       -88853.759             0.520            0.044   MAY BE DIVERGING... INSPECT ELBO
##   2100       -83242.743             0.474            0.044
##   2200       -84042.407             0.223            0.039
##   2300       -84309.521             0.053            0.037
##   2400       -84856.001             0.048            0.034
##   2500       -82544.820             0.036            0.034
##   2600       -83741.473             0.033            0.031
##   2700       -82740.379             0.033            0.031
##   2800       -84308.846             0.028            0.028
##   2900       -84333.718             0.025            0.027
##   3000       -83939.880             0.024            0.019
##   3100       -84139.269             0.022            0.014
##   3200       -82799.150             0.021            0.014
##   3300       -83281.625             0.021            0.014
##   3400       -83891.623             0.021            0.014
##   3500       -83244.158             0.020            0.014
##   3600       -82497.327             0.019            0.012
##   3700       -82496.565             0.017            0.010   MEDIAN ELBO CONVERGED
## Drawing a sample of size 2000 from the approximate posterior...
## COMPLETED.
## Finished in  0.8 seconds.
```

```r
# Extract summary
poisson_vi_summary <- fit_poisson_vi$summary()

# Print summary table
poisson_vi_summary
```

```
## # A tibble: 1,472 x 7
##    variable           mean      median         sd        mad         q5      q95
##    <chr>             <dbl>       <dbl>      <dbl>      <dbl>      <dbl>    <dbl>
##  1 lp__         -82446.     -82441.     141.       136.         -8.27e+4 -8.22e+4
##  2 lp_approx__     -4.97       -4.64     2.23       2.11       -9.13e+0 -1.97e+0
##  3 beta[1]          7.83        7.83     0.000984   0.000980    7.83e+0  7.83e+0
##  4 beta[2]          0.323       0.323    0.00129    0.00132     3.21e-1  3.26e-1
##  5 beta[3]          0.220       0.220    0.000691   0.000688    2.19e-1  2.22e-1
##  6 beta[4]          0.445       0.445    0.00183    0.00189     4.42e-1  4.48e-1
##  7 beta[5]          0.452       0.452    0.000375   0.000365    4.51e-1  4.53e-1
##  8 beta[6]          0.235       0.235    0.000543   0.000542    2.34e-1  2.36e-1
##  9 beta[7]         -0.0958     -0.0958   0.000517   0.000499   -9.66e-2 -9.49e-2
## 10 beta[8]         -0.0559     -0.0560   0.00113    0.00113    -5.78e-2 -5.41e-2
## # i 1,462 more rows
```

10. Compare Poisson HMC and VI

```r
poisson_hmc_beta <- fit_poisson_hmc$summary(variables = "beta")
poisson_vi_beta  <- fit_poisson_vi$summary(variables = "beta")

comparison_poisson <- data.frame(
  parameter = poisson_hmc_beta$variable,
  hmc_mean  = poisson_hmc_beta$mean,
  hmc_sd    = poisson_hmc_beta$sd,
  vi_mean   = poisson_vi_beta$mean,
  vi_sd     = poisson_vi_beta$sd
)

comparison_poisson
```

```
##    parameter    hmc_mean       hmc_sd     vi_mean        vi_sd
## 1    beta[1]  7.83911134 0.0021856435  7.83167741 0.0009839892
## 2    beta[2]  0.33154175 0.0023290504  0.32348294 0.0012929324
## 3    beta[3]  0.22769814 0.0029090617  0.22046135 0.0006911088
## 4    beta[4]  0.45176851 0.0020161697  0.44501448 0.0018317399
## 5    beta[5]  0.45962165 0.0011433077  0.45202753 0.0003753108
## 6    beta[6]  0.24295421 0.0010411229  0.23491647 0.0005425597
## 7    beta[7] -0.08695160 0.0006188522 -0.09578930 0.0005170358
## 8    beta[8] -0.06241207 0.0006118849 -0.05594871 0.0011308019
## 9    beta[9]  0.01873114 0.0012395911  0.01156136 0.0011505168
## 10  beta[10] -0.14658813 0.0037096590 -0.15317263 0.0032456575
```

11. Write Negative Binomial Stan model

```r
negbin_code <- "
data {
  int<lower=1> N;
  array[N] int<lower=0> y;
  int<lower=1> K;
  matrix[N, K] X;
}
parameters {
```

```
  vector[K] beta;
  real log_phi;
}
transformed parameters {
  real<lower=0> phi;
  phi = exp(log_phi);
}
model {
  beta ~ normal(0, 2.5);
  log_phi ~ normal(0, 1);
  y ~ neg_binomial_2_log(X * beta, phi);
}
generated quantities {
  vector[N] log_lik;
  array[N] int y_rep;
  for (n in 1:N) {
    log_lik[n] = neg_binomial_2_log_lpmf(y[n] | X[n] * beta, phi);
    y_rep[n] = neg_binomial_2_log_rng(X[n] * beta, phi);
  }
}
"

# Remove old file if it exists
if (file.exists("negbin_model.stan")) file.remove("negbin_model.stan")

# Write the Stan file
writeLines(negbin_code, "negbin_model.stan")

# Verify file contents
readLines("negbin_model.stan")
```

```
##  [1] ""
##  [2] "data {"
##  [3] "  int<lower=1> N;"
##  [4] "  array[N] int<lower=0> y;"
##  [5] "  int<lower=1> K;"
##  [6] "  matrix[N, K] X;"
##  [7] "}"
##  [8] "parameters {"
##  [9] "  vector[K] beta;"
## [10] "  real log_phi;"
## [11] "}"
## [12] "transformed parameters {"
## [13] "  real<lower=0> phi;"
## [14] "  phi = exp(log_phi);"
## [15] "}"
## [16] "model {"
## [17] "  beta ~ normal(0, 2.5);"
## [18] "  log_phi ~ normal(0, 1);"
## [19] "  y ~ neg_binomial_2_log(X * beta, phi);"
## [20] "}"
## [21] "generated quantities {"
## [22] "  vector[N] log_lik;"
```

```
## [23] "  array[N] int y_rep;"
## [24] "  for (n in 1:N) {"
## [25] "    log_lik[n] = neg_binomial_2_log_lpmf(y[n] | X[n] * beta, phi);"
## [26] "    y_rep[n] = neg_binomial_2_log_rng(X[n] * beta, phi);"
## [27] "  }"
## [28] "}"
## [29] ""
```

12. Compile Negative Binomial model

```r
nb_mod <- cmdstan_model("negbin_model.stan", force_recompile = TRUE)
```

13. Fit Negative Binomial model using HMC

```r
fit_nb_hmc <- nb_mod$sample(
  data = stan_data,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1500,
  iter_sampling = 1500,
  adapt_delta = 0.99,
  max_treedepth = 15,
  seed = 123,
  refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
```

```
## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1

## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
```

```
## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
```

```
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
```

```
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 4.4 seconds.
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 4.7 seconds.
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 finished in 4.9 seconds.
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 finished in 6.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 5.0 seconds.
## Total execution time: 6.0 seconds.
```

```r
# Extract summary
nb_hmc_summary <- fit_nb_hmc$summary()

# Print summary table
nb_hmc_summary
```

```
## # A tibble: 1,473 x 10
##    variable       mean     median     sd    mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>      <dbl>  <dbl>  <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__      -6141.     -6141.     2.40   2.20   -6.15e+3 -6.14e+3  1.00    2399.
##  2 beta[1]      7.83       7.83   0.0347 0.0336  7.77e+0  7.88e+0  1.00    1899.
##  3 beta[2]      0.309      0.308  0.0365 0.0365  2.50e-1  3.70e-1  1.00    2333.
##  4 beta[3]      0.177      0.177  0.0484 0.0485  9.55e-2  2.56e-1  1.00    2072.
##  5 beta[4]      0.464      0.464  0.0319 0.0321  4.12e-1  5.17e-1  1.00    3216.
##  6 beta[5]      0.482      0.482  0.0201 0.0207  4.48e-1  5.15e-1  1.00    5223.
##  7 beta[6]      0.304      0.304  0.0185 0.0184  2.74e-1  3.34e-1  1.00    2416.
##  8 beta[7]     -0.106     -0.106  0.0110 0.0110 -1.24e-1 -8.76e-2  1.00    5632.
##  9 beta[8]     -0.0745    -0.0745 0.0107 0.0107 -9.19e-2 -5.73e-2  1.00    5755.
## 10 beta[9]      0.0328     0.0329 0.0218 0.0220 -2.65e-3  6.88e-2  1.00    4491.
## # i 1,463 more rows
## # i 1 more variable: ess_tail <dbl>
```

Since there is warning, we need to check r-hat and ESS

```r
nb_hmc_summary <- fit_nb_hmc$summary()

max(nb_hmc_summary$rhat)
```

```
## [1] 1.004275
```

```r
min(nb_hmc_summary$ess_bulk)
```

```
## [1] 1899.109
```

14. Inspect Negative Binomial HMC diagnostics

```r
# Check the largest R-hat value
max(nb_hmc_summary$rhat, na.rm = TRUE)
```

```
## [1] 1.004275
```

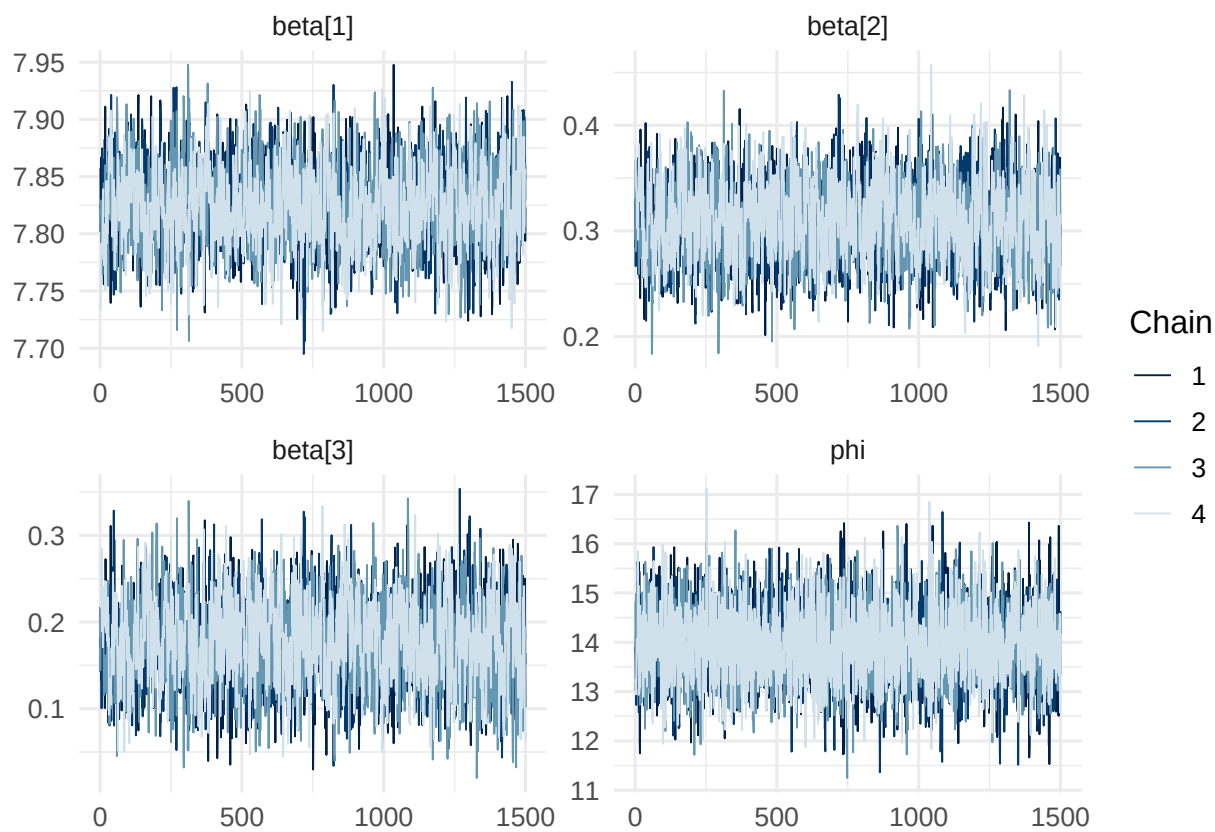
```r
# Check the smallest bulk ESS
min(nb_hmc_summary$ess_bulk, na.rm = TRUE)
```

```
## [1] 1899.109
```

```r
# Check the smallest tail ESS
min(nb_hmc_summary$ess_tail, na.rm = TRUE)
```

```
## [1] 2758.454
```

```r
# Traceplots for selected coefficients and dispersion parameter
mcmc_trace(
  fit_nb_hmc$draws(variables = c("beta[1]", "beta[2]", "beta[3]", "phi"))
)
```




15. Fit Negative Binomial model using Variational Inference

```r
fit_nb_vi <- nb_mod$variational(
  data = stan_data,
  algorithm = "meanfield",
  iter = 20000,
  output_samples = 2000,
  seed = 123
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 5.5e-05 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 0.55 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100        -9180.183             1.000            1.000
##    200        -8671.916             0.529            1.000
##    300        -7988.055             0.381            0.086
##    400        -6902.279             0.325            0.157
##    500        -6356.607             0.277            0.086
##    600        -6233.192             0.235            0.086
##    700        -6194.169             0.202            0.086
##    800        -6204.706             0.177            0.086
##    900        -6192.126             0.157            0.059
##   1000        -6211.608             0.142            0.059
##   1100        -6201.880             0.129            0.020
##   1200        -6197.756             0.119            0.020
##   1300        -6233.682             0.110            0.006   MEDIAN ELBO CONVERGED
## Drawing a sample of size 2000 from the approximate posterior...
## COMPLETED.
## Finished in  1.0 seconds.
```

```r
# Extract summary
nb_vi_summary <- fit_nb_vi$summary()

# Print summary table
nb_vi_summary
```

```
## # A tibble: 1,474 x 7
##    variable          mean     median       sd      mad         q5        q95
##    <chr>            <dbl>      <dbl>    <dbl>    <dbl>      <dbl>      <dbl>
##  1 lp__         -6205.     -6203.     15.1     14.6     -6232.     -6182.
##  2 lp_approx__    -5.43      -5.11     2.32     2.13       -9.60      -2.31
```

```
##  3 beta[1]         7.85       7.85    0.00913  0.00912     7.84       7.87
##  4 beta[2]         0.333      0.333   0.0164   0.0164      0.307      0.361
##  5 beta[3]         0.208      0.208   0.0219   0.0215      0.172      0.244
##  6 beta[4]         0.501      0.502   0.0222   0.0218      0.465      0.538
##  7 beta[5]         0.507      0.507   0.0142   0.0142      0.483      0.530
##  8 beta[6]         0.332      0.332   0.0124   0.0125      0.312      0.353
##  9 beta[7]        -0.119     -0.119   0.0136   0.0137     -0.141     -0.0965
## 10 beta[8]        -0.0661    -0.0663  0.0109   0.0108     -0.0839    -0.0484
## # i 1,464 more rows
```

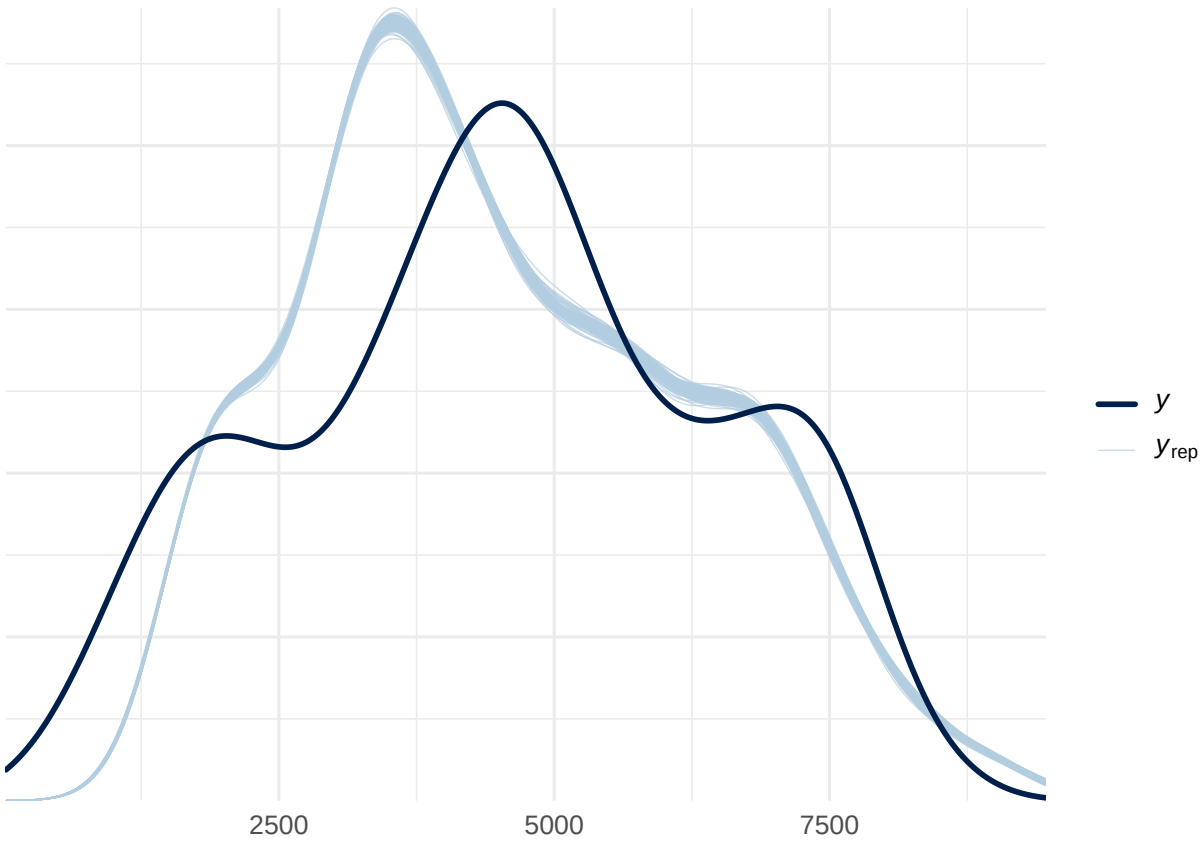
16. Compare Negative Binomial HMC and VI

```r
nb_hmc_beta <- fit_nb_hmc$summary(variables = c("beta", "phi"))
nb_vi_beta  <- fit_nb_vi$summary(variables = c("beta", "phi"))

comparison_nb <- data.frame(
  parameter = nb_hmc_beta$variable,
  hmc_mean  = nb_hmc_beta$mean,
  hmc_sd    = nb_hmc_beta$sd,
  vi_mean   = nb_vi_beta$mean,
  vi_sd     = nb_vi_beta$sd
)

comparison_nb
```

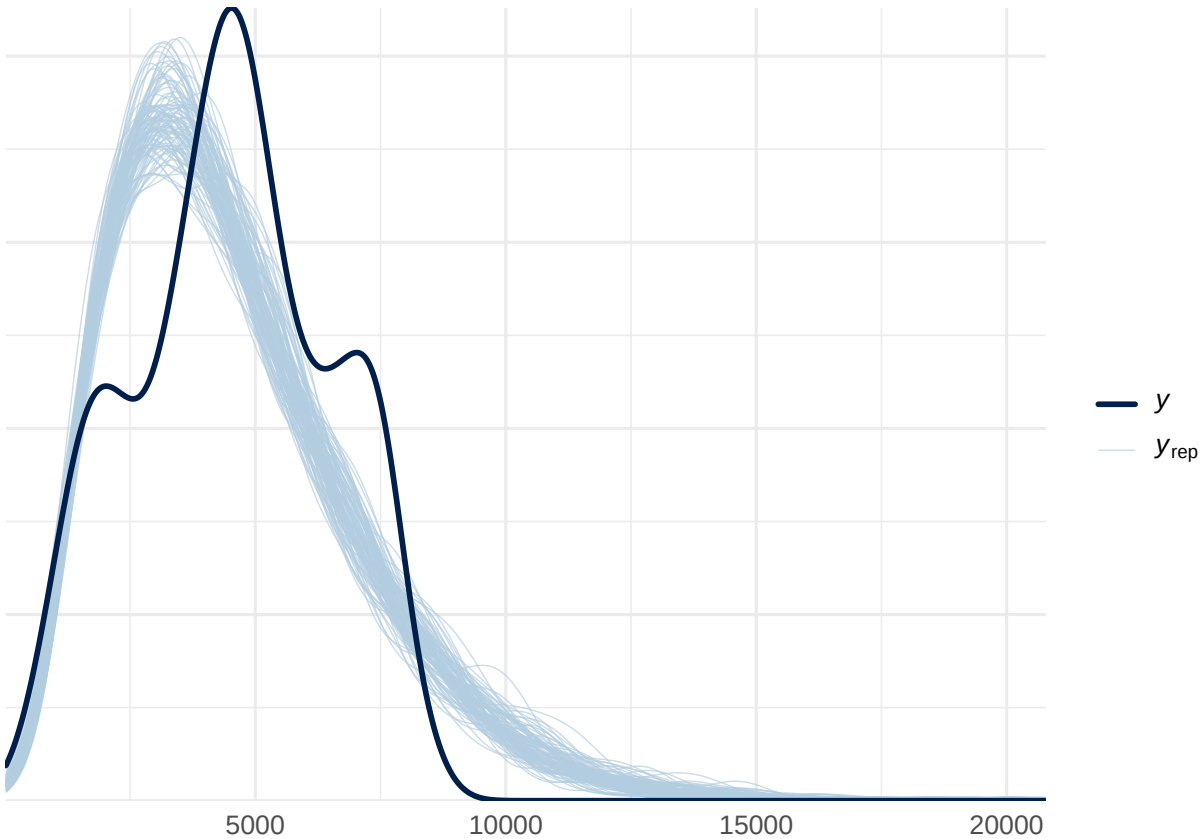
```
##    parameter    hmc_mean     hmc_sd     vi_mean       vi_sd
## 1    beta[1]  7.82624806 0.03474125  7.85464401 0.009132069
## 2    beta[2]  0.30908460 0.03646106  0.33341360 0.016411603
## 3    beta[3]  0.17654453 0.04835608  0.20755368 0.021862086
## 4    beta[4]  0.46387628 0.03186679  0.50145743 0.022186716
## 5    beta[5]  0.48165124 0.02014094  0.50677473 0.014215533
## 6    beta[6]  0.30401928 0.01845978  0.33191739 0.012422683
## 7    beta[7] -0.10581898 0.01104960 -0.11910159 0.013586677
## 8    beta[8] -0.07453996 0.01066716 -0.06613203 0.010944632
## 9    beta[9]  0.03280309 0.02178496  0.06454817 0.010212477
## 10  beta[10] -0.17016123 0.06123285 -0.14979639 0.067730173
## 11       phi 13.94255775 0.74569683 13.38369343 0.708746702
```

17. Posterior predictive checks

```r
# Extract posterior predictive draws
yrep_poisson <- as_draws_matrix(fit_poisson_hmc$draws("y_rep"))
yrep_nb <- as_draws_matrix(fit_nb_hmc$draws("y_rep"))

# Posterior predictive density overlay
ppc_dens_overlay(y = bike_clean$cnt, yrep = yrep_poisson[1:100, ])
```




```
ppc_dens_overlay(y = bike_clean$cnt, yrep = yrep_nb[1:100, ])
```




```r
# Compare observed and replicated mean and variance
ppc_stats <- data.frame(
  model = c("Observed", "Poisson_HMC", "NB_HMC"),
  mean = c(
    mean(bike_clean$cnt),
    mean(rowMeans(yrep_poisson)),
    mean(rowMeans(yrep_nb))
  ),
  variance = c(
    var(bike_clean$cnt),
    mean(apply(yrep_poisson, 1, var)),
    mean(apply(yrep_nb, 1, var))
  )
)

ppc_stats
```

```
##         model     mean variance
## 1    Observed 4508.007  3748141
## 2 Poisson_HMC 4508.043  3143307
## 3      NB_HMC 4555.502  5731131
```

18. Save all summary tables

```r
write.csv(as.data.frame(poisson_hmc_summary), "poisson_hmc_summary.csv", row.names = FALSE)
write.csv(as.data.frame(poisson_vi_summary), "poisson_vi_summary.csv", row.names = FALSE)
write.csv(as.data.frame(nb_hmc_summary), "nb_hmc_summary.csv", row.names = FALSE)
write.csv(as.data.frame(nb_vi_summary), "nb_vi_summary.csv", row.names = FALSE)

write.csv(comparison_poisson, "comparison_poisson.csv", row.names = FALSE)
write.csv(comparison_nb, "comparison_nb.csv", row.names = FALSE)
write.csv(ppc_stats, "ppc_stats.csv", row.names = FALSE)
```

19. Compare computation times

```r
fit_poisson_hmc$time()
```

```
## $total
## [1] 1.809298
##
## $chains
##   chain_id warmup sampling total
## 1        1  0.669    1.005 1.674
## 2        2  0.587    0.929 1.516
## 3        3  0.580    0.912 1.492
## 4        4  0.581    1.028 1.609
```

```r
fit_poisson_vi$time()
```

```
## $total
## [1] 0.7630591
```

```r
fit_nb_hmc$time()
```

```
## $total
## [1] 6.042777
##
## $chains
##   chain_id warmup sampling total
## 1        1  2.307    2.640 4.947
## 2        2  2.231    2.491 4.722
## 3        3  2.502    3.501 6.003
## 4        4  2.011    2.439 4.450
```

```r
fit_nb_vi$time()
```

```
## $total
## [1] 0.9646602
```